

Document Number: P0482R0
Date: 2016-10-17
Audience: Evolution Working Group
Library Evolution Working Group
Reply-to: Tom Honermann <tom@honermann.net>

char8_t: A type for UTF-8 characters and strings

- [Introduction](#)
- [Motivation](#)
- [Design Considerations](#)
 - [Backward compatibility](#)
 - [Core language backward compatibility features](#)
 - [Implicit conversions from UTF-8 strings to ordinary strings](#)
 - [Library backward compatibility features](#)
 - [Implicit conversion from std::u8string to std::string](#)
 - [Deduced types for UTF-8 literals](#)
 - [Should UTF-8 literals continue to be referred to as narrow literals?](#)
 - [What should be the underlying type of char8_t?](#)
 - [Deprecated features](#)
 - [codecvt and codecvt_byname specializations](#)
 - [u8path path factory functions](#)
- [Implementation Experience](#)
- [Formal Wording](#)
 - [Core Wording](#)
 - [Library Wording](#)
- [Acknowledgements](#)
- [References](#)

Introduction

C++11 introduced support for UTF-8, UTF-16, and UTF-32 encoded string literals via [N2249](#) ^[N2249]. New `char16_t` and `char32_t` types were added to hold values of code units for the UTF-16 and UTF-32 variants, but a new type was not added for the UTF-8 variants. Instead, UTF-8 character literals (added in C++17 via [N4197](#) ^[N4197]) and string literals were defined in terms of the `char` type used for the code unit type of ordinary character and string literals. UTF-8 is the only text encoding mandated to be supported by the C++ standard for which there is no distinct code unit type. Lack of a distinct type for UTF-8 encoded character and string literals prevents the use of overloading and template specialization in interfaces designed for interoperability with encoded text. The inability to infer an encoding for narrow characters and strings limits design possibilities and hinders the production of elegant interfaces that work seamlessly in generic code. Library authors must choose to limit encoding support, design interfaces that require users to explicitly specify encodings, or provide distinct interfaces for, at least, the implementation defined execution and UTF-8 encodings.

Whether `char` is a signed or unsigned type is implementation defined and implementations that use an 8-bit signed `char` are at a disadvantage with respect to working with UTF-8 encoded text due to the necessity of having to rely on conversions to unsigned types in order to correctly process leading and continuation code units of multi-byte encoded code points.

The lack of a distinct type and the use of a code unit type with a range that does not portably include the full unsigned range of UTF-8 code units presents challenges for working with UTF-8 encoded text that are not present when working with UTF-16 or UTF-32 encoded text. Enclosed is a proposal for a new `char8_t` fundamental type and related library enhancements intended to remove barriers to working with UTF-8 encoded text and to enable generic interfaces that work with all five of the standard mandated text encodings in a consistent manner.

This proposal is incomplete as the author ran out of time preparing it for the Issaquah mailing deadline. The following are known deficiencies that are expected to be addressed in a future revision of this proposal.

- Backward compatibility is not adequately addressed. There is some discussion in the design considerations section, but no provisions addressing backward compatibility are currently present in the wording. The proposed changes effectively bring the standard to the state the author feels it would likely be in had `char8_t` been added at the same time as `char16_t` and `char32_t` were.
- An implementation of the proposed changes is not yet available for assessing the impact to backward compatibility.

- The claim that a new type may allow compilers to better optimize code that works with UTF-8 strings is unsubstantiated.
- Wording updates for clauses C and D have not yet been provided.
- Impact to other proposals such as [P0353R0](#) [\[P0353R0\]](#) is not discussed.

Motivation

Consider the following string literal expressions, all of which encode U+0123, LATIN SMALL LETTER G WITH CEDILLA:

```
u8"\u0123" // UTF-8:  const char[]:      0xC4 0xA3 0x00
u"\u0123"  // UTF-16: const char16_t[]: 0x0123 0x0000
U"\u0123"  // UTF-32: const char32_t[]: 0x00000123 0x00000000
"\u0123"   // ????:  const char[]:      ???
L"\u0123"  // ????:  const wchar_t[]:    ???
```

The UTF-8, UTF-16, and UTF-32 string literals have well-defined and portable sequences of code unit values. The ordinary and wide string literal code unit sequences depend on the implementation defined execution and execution wide encodings respectively. Code that is designed to work with text encodings must be able to differentiate these strings. This is straight forward for wide, UTF-16, and UTF-32 string literals since they each have a distinct code unit type suitable for differentiation via function overloading or template specialization. But for ordinary and UTF-8 string literals, differentiating between them requires additional information since they have the same code unit type. That additional information might be provided implicitly via differently named functions, or explicitly via additional function or template arguments. For example:

```
// Differentiation by function name:
void do_x(const char *);
void do_x_utf8(const char *);

// Differentiation by suffix for user-defined literals:
int operator ""_udl(const char *s, std::size_t);
int operator ""_udl_utf8(const char *s, std::size_t);

// Differentiation by function parameter:
void do_x(const char *, bool is_utf8);

// Differentiation by template parameter:
template<bool IsUTF8>
void do_x(const char *);
```

The requirement to, in some way, specify the text encoding, other than through the type of the string, limits the ability to provide elegant encoding sensitive interfaces. Consider the following invocations of the `make_text_view` function proposed in [P0244R1](#) [\[P0244R1\]](#):

```
make_text_view<execution_character_encoding>("text")
make_text_view<execution_wide_character_encoding>(L"text")
make_text_view<utf8_encoding>(u8"text")
make_text_view<utf16_encoding>(u"text")
make_text_view<utf32_encoding>(U"text")
```

For each invocation, the encoding of the string literal is known at compile time, so having to explicitly specify the encoding tag feels redundant. If UTF-8 strings had a distinct type, then the encoding type could be inferred, while still allowing an overriding tag to be supplied:

```
make_text_view("text")    // defaults to execution_character_encoding.
make_text_view(L"text")   // defaults to execution_wide_character_encoding.
make_text_view(u8"text")  // defaults to utf8_encoding.
make_text_view(u"text")   // defaults to utf16_encoding.
make_text_view(U"text")   // defaults to utf32_encoding.
make_text_view<utf16be_encoding>("\0t\0e\0x\0t\0") // Default overridden.
```

The inability to infer an encoding for narrow strings doesn't just limit the interfaces of new features under consideration. Compromised interfaces are already present in the standard library.

Consider the design of the `codecvt` class template. The standard specifies the following specializations of `codecvt` be provided to enable transcoding text from one encoding to another.

```
codecvt<char, char, mbstate_t>    // #1
```

```
codecvt<wchar_t, char, mbstate_t> // #2
codecvt<char16_t, char, mbstate_t> // #3
codecvt<char32_t, char, mbstate_t> // #4
```

#1 performs no conversions. #2 converts between strings encoded in the implementation defined wide and narrow encodings. #3 and #4 convert between either the UTF-16 or UTF-32 encoding and the UTF-8 encoding. Specializations are not currently specified for conversion between the implementation defined narrow and wide encodings and any of the UTF-8, UTF-16, or UTF-32 encodings. However, if support for such conversions were to be added, the desired interfaces are already taken by #1, #3 and #4.

The file system interface adopted for C++17 via [P0218R1](#) [\[P0218R1\]](#) provides an example of a feature that supports all five of the standard mandated encodings, but does so with an asymmetric interface due to the inability to overload functions for UTF-8 encoded strings. Class `std::filesystem::path` provides the following constructors to initialize a path object based on a range of code unit values where the encoding is inferred based on the value type of the range.

```
template <class Source>
path(const Source& source);
template <class InputIterator>
path(InputIterator first, InputIterator last);
```

§ 27.10.8.2.2 [path.type.cvt] describes how the source encoding is determined based on whether the source range value type is `char`, `wchar_t`, `char16_t`, or `char32_t`. A range with value type `char` is interpreted using the implementation defined narrow execution encoding. It is not possible to construct a path object from UTF-8 encoded text using these constructors.

To accommodate UTF-8 encoded text, the file system library specifies the following factory functions. Matching factory functions are not provided for other encodings.

```
template <class Source>
path u8path(const Source& source);
template <class InputIterator>
path u8path(InputIterator first, InputIterator last);
```

The requirement to construct `path` objects using one interface for UTF-8 strings vs another interface for all other supported encodings creates unnecessary difficulties for portable code. Consider an application that uses UTF-8 as its internal encoding on POSIX systems, but uses UTF-16 on Windows. Conditional compilation or other abstractions must be implemented and used in otherwise platform neutral code to construct `path` objects.

The inability to infer an encoding based on string type is not the only challenge posed by use of `char` as the UTF-8 code unit type. The following code exhibits implementation defined behavior.

```
bool is_utf8_multibyte_code_unit(char c) {
    return c >= 0x80;
}
```

UTF-8 leading and continuation code units have values in the range 128 (0x80) to 255 (0xFF). In the common case where `char` is implemented as a signed 8-bit type with a two's complement representation and a range of -128 (-0x80) to 127 (0x7F), these values exceed the unsigned range of the `char` type. Such implementations typically encode such code units as unsigned values which are then reinterpreted as signed values when read. In the code above, integral promotion rules result in `c` being promoted to type `int` for comparison to the `0x80` operand. If `c` holds a value corresponding to a leading or continuation code unit value, then its value will be interpreted as negative and the promoted value of type `int` will likewise be negative. The result is that the comparison is always false for these implementations.

To correct the code above, explicit conversions are required. For example:

```
bool is_utf8_multibyte_code_unit(char c) {
    return static_cast<unsigned char>(c) >= 0x80;
}
```

Finally, processing of UTF-8 strings is currently subject to an optimization pessimization due to glvalue expressions of type `char` potentially aliasing objects of other types. Use of a distinct type that does not share this aliasing behavior may allow for further compiler optimizations.

Design Considerations

Backward compatibility

This proposal does not specify any backward compatibility features other than to retain interfaces that it deprecates. The lack of such features is not due to a belief that backward compatibility features are not necessary. The author believes such features are necessary, but time constraints prevented adequately researching what issues must be addressed, to what degree they must be addressed, and how those features should be specified. The author intends to address these concerns in a future revision of this document. In the meantime, the following sections discuss some of the backward compatibility impact and possible solution directions.

Core language backward compatibility features

Implicit conversions from UTF-8 strings to ordinary strings

It may be necessary to allow implicit conversions for UTF-8 string literals from `const char8_t[]` to `const char[]` to allow currently well-formed code like the following to remain well-formed:

```
template<typename T> void f(const T*);
void f(const char*);
f(u8"text");           // Ok, calls f(const char*).
...
char u8a[] = u8"text"; // Ok.
const char (&u8r)[] = u8"text"; // Ok.
const char *u8s = u8"text"; // Ok.
```

It may also be necessary to permit implicit conversions for non-literal UTF-8 strings:

```
const auto *u8s = u8"text"; // C++14: Ok, type deduced to const char*.
                               // This proposal: Ok, type deduced to const char8_t*.
const char *s = u8s;         // C++14: Ok, u8s has type const char*.
                               // This proposal: An implicit conversion from const char8_t*
                               // to const char* would be required for this assignment
                               // to remain well-formed.
```

If such implicit conversions are found to be necessary, specifying them may present a small challenge. The standard conversion sequence might have to be modified to allow a data representation conversion prior to an lvalue transformation in order for an argument of, for example, array of `char8_t` to match a parameter of type `char*`. However, the standard conversion sequence, as described in § 13.3.3.1.1 [over.ics.scs], states that lvalue transformations, including the array-to-pointer conversion, are performed before promotions and conversions that might change the data representation. It may be feasible to avoid such a change by stating that a candidate function that involves such an implicit conversion is only a viable function if no other viable non-template functions are identified, but the author has not yet convinced himself of this possibility.

If such implicit conversions are found to be necessary, providing them as deprecated features would enable a transition period and eventual removal.

Library backward compatibility features

Implicit conversion from `std::u8string` to `std::string`

This proposal includes a new specialization of `std::basic_string` for the new `char8_t` type, the associated typedef `std::u8string`, and changes to several functions to now return `std::u8string` instead of `std::string`. This change renders ill-formed the following code that is currently well-formed.

```
void f(std::filesystem::path p) {
    std::string s = p.u8string(); // C++14: Ok.
                                   // This proposal: ill-formed unless conversions
                                   // from std::u8string to std::string
                                   // are provided.
}
```

Implicit conversions from `std::u8string` to `std::string` would be undesirable in general. If they are found to be necessary, providing them as a deprecated feature seems warranted.

Deduced types for UTF-8 literals

Under this proposal, UTF-8 string and character literals have type `const char8_t[]` and `char8_t` respectively. This affects the types deduced for placeholder types and template parameter types.

```
template<typename T1, typename T2>
void ft(T1, T2);
...
ft(u8"text", u8'c'); // C++14: T1 deduced to const char*, T2 deduced to char.
                    // This proposal: T1 deduced to const char8_t*, T2 deduced to char8_t.
...
auto u8s = u8"text"; // C++14: Type deduced to const char*.
                    // This proposal: Type deduced to const char8_t*.
auto u8c = u8'c';    // C++14: Type deduced to char.
                    // This proposal: Type deduced to char8_t.
```

This has the potential to affect backward compatibility in code that depends on overload resolution selecting the same overload for calls involving both ordinary and UTF-8 strings. For example:

```
template<typename T>
void ft(T) {
    static int count = 0;
    return count++;
}
...
ft("text"); // Returns 0.
ft(u8"text"); // C++14: Returns 1.
              // This proposal: Returns 0.
```

Should UTF-8 literals continue to be referred to as narrow literals?

UTF-8 literals are maintained as narrow literals in this proposal.

What should be the underlying type of char8_t?

There are several choices for the underlying type of `char8_t`. Use of `unsigned char` closely aligns with historical use. Use of `uint_least8_t` would maintain consistency with how the underlying types of `char16_t` and `char32_t` are specified.

This proposal specifies `unsigned char` as the underlying type as noted in the changes to § 3.9.1 [basic.fundamental] paragraph 5.

Deprecated features

codecvt and codecvt_byname specializations

This proposal introduces new `codecvt` and `codecvt_byname` specializations that use `char8_t` for conversion to and from UTF-8 and deprecates the existing ones specified in terms of `char`. The new specializations are functionally identical to the deprecated ones.

u8path path factory functions

Filesystem path objects may now be constructed with UTF-8 strings using the existing path constructors used for construction with other encodings as specified in § 27.10.8.2.2 [path.type.cvt] and § 27.10.8.4.1 [path.construct]. This proposal deprecates the existing `u8path` path factory functions specified in § 27.10.8.6.2 [path.factory].

Implementation Experience

None yet, but the author intends to prototype an implementation in `gcc/libstdc++` and/or `Clang/libc++`.

Formal Wording

☐ Hide deleted text

These changes are relative to [N4606](#) [\[N4606\]](#)

Core Wording

Add `char8_t` to the list of keywords in table 3 in 2.11 [lex.key] paragraph 1.

Change in 2.13.3 [lex.ccon] paragraph 3:

A character literal that begins with `u8`, such as `u8'w'`, is a character literal of type `char`**`char8_t`**, known as a *UTF-8 character literal*. [...]

Remove 2.13.5 [lex.string] paragraph 7:

~~A string literal that begins with `u8`, such as `u8"asdf"`, is a UTF-8 string literal.~~

Change in 2.13.5 [lex.string] paragraph 8:

Ordinary string literals and UTF-8 string literals are also referred to as narrow string literals. ~~A narrow string literal has type "array of `n` `const char`", where `n` is the size of the string as defined below, and has static storage duration (3.7).~~

Add a new paragraph after 2.13.5 [lex.string] paragraph 8:

An ordinary string literal has type "array of `n` `const char`", where `n` is the size of the string as defined below, and has static storage duration (3.7).

Change in 2.13.5 [lex.string] paragraph 9:

~~For a UTF-8 string literal, each successive element of the object representation (3.9) has the value of the corresponding code unit of the UTF-8 encoding of the string.~~ **A string literal that begins with `u8`, such as `u8"asdf"`, is a UTF-8 string literal, also referred to as a `char8_t` string literal. A `char8_t` string literal has type "array of `n` `const char8_t`", where `n` is the size of the string as defined below; each successive element of the object representation (3.9) has the value of the corresponding code unit of the UTF-8 encoding of the *s-char-sequence*. A single *s-char* may produce more than one `char8_t` code unit.**

Change in 2.13.5 [lex.string] paragraph 15:

[...] In a narrow string literal, a *universal-character-name* may map to more than one `char` **or `char8_t`** element due to multibyte encoding. [...]

Change in 3.9.1 [basic.fundamental] paragraph 1:

Objects declared ~~as characters~~**with type `(char)`** shall be large enough to store any member of the implementation's basic character set. If a character from this set is stored in a character object, the integral value of that character object is equal to the value of the single character literal form of that character. It is implementation-defined whether a `char` object can hold negative values. Characters **declared with type `char`** can be explicitly declared unsigned or signed. Plain `char`, signed `char`, and unsigned `char` are three distinct types, collectively called ~~narrow~~**ordinary** character types. **The ordinary character types and `char8_t` are collectively called narrow character types.** A `char`, a signed `char`, ~~and an unsigned char,~~ **and a `char8_t`** occupy the same amount of storage and have the same alignment requirements (3.11); that is, they have the same object representation. For narrow character types, all bits of the object representation participate in the value representation. [*Note*: A bit-field of narrow character type whose length is larger than the number of bits in the object representation of that type has padding bits; see 9.2.4. — *end note*] For unsigned narrow character types, **including `char8_t`**, each possible bit pattern of the value representation represents a distinct number. These requirements do not hold for other types. In any particular implementation, a plain `char` object ~~can~~ **shall** take on either the same values as a signed `char` or an unsigned `char`; which one is implementation-defined. For each value *i* of type unsigned `char`, **or `char8_t`** in the range 0 to 255 inclusive, there exists a value *j* of type `char` such that the result of an integral conversion (4.8) from *i* to `char` is *j*, and the result of an integral conversion from *j* to unsigned `char` **or `char8_t`** is *i*.

Change in 3.9.1 [basic.fundamental] paragraph 5:

[...] Type `wchar_t` shall have the same size, signedness, and alignment requirements (3.11) as one of the other integral types, called its underlying type. **Type `char8_t` denotes a distinct type with the same size, signedness, and alignment as unsigned `char`, called its underlying type.** Types `char16_t` and `char32_t` denote distinct types with the same size, signedness, and alignment as `uint_least16_t` and `uint_least32_t`, respectively, in `<cstdint>`, called the underlying types.

Change in 3.9.1 [basic.fundamental] paragraph 7:

Types `bool`, `char`, **`char8_t`**, `char16_t`, `char32_t`, `wchar_t`, and the signed and unsigned integer types are collectively

called integral types.

Change in 4.15 [conv.rank] paragraph 1:

[...]
(1.8) — The ranks of **char8_t**, char16_t, char32_t, and wchar_t shall equal the ranks of their underlying types (3.9.1).
[...]

Change to footnote 62 associated with 5 [expr] paragraph 11 (11.5):

As a consequence, operands of type bool, **char8_t**, char16_t, char32_t, wchar_t, or an enumerated type are converted to some integral type.

Change in 5.3.3 [expr.sizeof] paragraph 1:

[...] sizeof(char), sizeof(signed char), ~~and~~ sizeof(unsigned char), **and sizeof(char8_t) are 1.** [...]

Change in 7.1.7.2 [dcl.type.simple] paragraph 1:

The simple type specifiers are

simple-type-specifier:

[...]
char
char8_t
char16_t
char32_t
[...]

Change in table 9 of 7.1.7.2 [dcl.type.simple] paragraph 4:

[...]
(4.5) — otherwise, decltype(e) is the type of e.
Table 9 — *simple-type-specifiers* and the types they specify

Specifier(s)	Type
[...]	[...]
char	"char"
unsigned char	"unsigned char"
signed char	"signed char"
char8_t	"char8_t"
char16_t	"char16_t"
char32_t	"char32_t"
[...]	[...]

[...]

Change in 8.6 [dcl.init] paragraph 17:

[...]
(17.3) — If the destination type is an array of characters, **an array of char8_t**, an array of char16_t, an array of char32_t, or an array of wchar_t, and the initializer is a string literal, see 8.6.2.
[...]

Change in 8.6.2 [dcl.init.string] paragraph 1:

An array of ~~narrow~~**ordinary** character type (3.9.1), **char8_t array**, char16_t array, char32_t array, or wchar_t array can be initialized by a narrow string literal, **char8_t string literal**, char16_t string literal, char32_t string literal, or wide string literal, respectively, [...]

Drafting note: It is intentional that an array of ordinary character type can be initialized by a narrow string literal, including UTF-8 string literals. This is a backward compatibility feature.

Change in 13.5.8 [over.literal] paragraph 3:

The declaration of a literal operator shall have a *parameter-declaration-clause* equivalent to one of the following:

```
[...]  
char  
wchar_t  
char8_t  
char16_t  
char32_t  
const char*, std::size_t  
const wchar_t*, std::size_t  
const char8_t*, std::size_t  
const char16_t*, std::size_t  
const char32_t*, std::size_t  
[...]
```

Library Wording

Change in 17.1 [library.general] paragraph 7:

The strings library (Clause 21) provides support for manipulating text represented as sequences of type `char`, **sequences of type `char8_t`**, sequences of type `char16_t`, sequences of type `char32_t`, sequences of type `wchar_t`, and sequences of any other character-like type.

Change in 17.3.3 [defs.character] paragraph 3:

```
[...]  
[ Note: The term does not mean only char, char8_t, char16_t, char32_t, and wchar_t objects, but any value that can be  
represented by a type that provides the definitions specified in these Clauses. — end note ]
```

Change in 18.3.2.2 [limits.syn]:

```
[...]  
template<> class numeric_limits<char>;  
template<> class numeric_limits<signed char>;  
template<> class numeric_limits<unsigned char>;  
template<> class numeric_limits<char8_t>;  
template<> class numeric_limits<char16_t>;  
template<> class numeric_limits<char32_t>;  
template<> class numeric_limits<wchar_t>;  
[...]
```

Change in 20.14 [function.objects] paragraph 2:

```
[...]  
// Hash function specializations  
template <> struct hash<bool>;  
template <> struct hash<char>;  
template <> struct hash<signed char>;  
template <> struct hash<unsigned char>;  
template <> struct hash<char8_t>;  
template <> struct hash<char16_t>;  
template <> struct hash<char32_t>;  
template <> struct hash<wchar_t>;  
[...]
```

Change in 20.14.14 [unord.hash] paragraph 1:

```
[...]  
template <> struct hash<bool>;  
template <> struct hash<char>;  
template <> struct hash<signed char>;  
template <> struct hash<unsigned char>;  
template <> struct hash<char8_t>;  
template <> struct hash<char16_t>;  
template <> struct hash<char32_t>;
```



```
template <> struct hash<wchar_t>;  
[...]
```

Change in 21.2 [char.traits] paragraph 1:

This subclause defines requirements on classes representing *character traits*, and defines a class template `char_traits<charT>`, along with **four** specializations, `char_traits<char>`, **`char_traits<char8_t>`**, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`, that satisfy those requirements.

Change in 21.2 [char.traits] paragraph 4:

This subclause specifies a class template, `char_traits<charT>`, and **four** explicit specializations of it, `char_traits<char>`, **`char_traits<char8_t>`**, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`, all of which appear in the header `<string>` and satisfy the requirements below.

Drafting note: 21.2p4 appears to unnecessarily duplicate information previously presented in 21.2p1.

Change in 21.2.3 [char.traits.specializations]:

```
namespace std {  
    template<> struct char_traits<char>;  
    template<> struct char_traits<char8_t>;  
    template<> struct char_traits<char16_t>;  
    template<> struct char_traits<char16_t>;  
    template<> struct char_traits<char32_t>;  
    template<> struct char_traits<wchar_t>;  
}
```

Change in 21.2.3 [char.traits.specializations] paragraph 1:

The header `<string>` shall define **four** specializations of the class template `char_traits`: `char_traits<char>`, **`char_traits<char8_t>`**, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`.

Add a new subclause after 21.2.3.1 [char.traits.specializations.char]:

```
21.2.3.X struct char_traits<char8_t> [char.traits.specializations.char8_t]  
namespace std {  
    template<> struct char_traits<char8_t> {  
        using char_type = char8_t;  
        using int_type = unsigned int;  
        using off_type = streamoff;  
        using pos_type = u8streampos;  
        using state_type = mbstate_t;  
        static void assign(char_type& c1, const char_type& c2) noexcept;  
        static constexpr bool eq(char_type c1, char_type c2) noexcept;  
        static constexpr bool lt(char_type c1, char_type c2) noexcept;  
        static int compare(const char_type* s1, const char_type* s2, size_t n);  
        static size_t length(const char_type* s);  
        static const char_type* find(const char_type* s, size_t n,  
        const char_type& a);  
        static char_type* move(char_type* s1, const char_type* s2, size_t n);  
        static char_type* copy(char_type* s1, const char_type* s2, size_t n);  
        static char_type* assign(char_type* s, size_t n, char_type a);  
        static constexpr int_type not_eof(int_type c) noexcept;  
        static constexpr char_type to_char_type(int_type c) noexcept;  
        static constexpr int_type to_int_type(char_type c) noexcept;  
        static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;  
        static constexpr int_type eof() noexcept;  
    };  
}
```

Add a new paragraph:

The type `u8streampos` shall be an implementation-defined type that satisfies the requirements for `pos_type` in 27.2.2

and 27.3.

Add another new paragraph:

The two-argument members `assign`, `eq`, and `lt` shall be defined identically to the built-in operators `=`, `==`, and `<` respectively.

Add another new paragraph:

The member `eof()` shall return an implementation-defined constant that cannot appear as a valid UTF-8 code unit.

Change in 21.3 [string.classes] paragraph 1:

The header `<string>` defines the `basic_string` class template for manipulating varying-length sequences of char-like objects and ~~four~~**five** *typedef-names*, `string`, `u8string`, `u16string`, `u32string`, and `wstring`, that name the specializations `basic_string<char>`, `basic_string<char8_t>`, `basic_string<char16_t>`, `basic_string<char32_t>`, and `basic_string<wchar_t>`, respectively.

Header `<string>` synopsis

```
#include <initializer_list>

namespace std {

    // 21.2, character traits:
    template<class charT> struct char_traits;
    template<> struct char_traits<char>;
    template<> struct char_traits<char8_t>;
    template<> struct char_traits<char16_t>;
    template<> struct char_traits<char32_t>;
    template<> struct char_traits<wchar_t>;
    [...]
    // basic_string typedef names
    using string = basic_string<char>;
    using u8string = basic_string<char8_t>;
    using u16string = basic_string<char16_t>;
    using u32string = basic_string<char32_t>;
    using wstring = basic_string<wchar_t>;
    [...]
    // 21.3.4, hash support:
    template<class T> struct hash;
    template<> struct hash<string>;
    template<> struct hash<u8string>;
    template<> struct hash<u16string>;
    template<> struct hash<u32string>;
    template<> struct hash<wstring>;

    namespace pmr {
        template <class charT, class traits = char_traits<charT>>
            using basic_string =
                std::basic_string<charT, traits, polymorphic_allocator<charT>>;
        using string = basic_string<char>;
        using u8string = basic_string<char8_t>;
        using u16string = basic_string<char16_t>;
        using u32string = basic_string<char32_t>;
        using wstring = basic_string<wchar_t>;
    }

    inline namespace literals {
        inline namespace string_literals {
            // 21.3.5, suffix for basic_string literals:
```

```

string operator "" s(const char* str, size_t len);
u8string operator "" s(const char8_t* str, size_t len);
u16string operator "" s(const char16_t* str, size_t len);
u32string operator "" s(const char32_t* str, size_t len);
wstring operator "" s(const wchar_t* str, size_t len);
}
}
}

```

Change in 21.3.4 [basic.string.hash]:

```

template<> struct hash<string>;
template<> struct hash<u8string>;
template<> struct hash<u16string>;
template<> struct hash<u32string>;
template<> struct hash<wstring>;

```

Add a new paragraph after 21.3.5 [basic.string.literals] paragraph 1:

```

u8string operator "" s(const char8_t* str, size_t len);
Returns: u8string{str, len}.

```

Change in 21.4.1 [string.view.synop]:

```

[...]
// basic_string_view typedef names
using string_view = basic_string_view<char>;
using u8string_view = basic_string_view<char8_t>;
using u16string_view = basic_string_view<char16_t>;
using u32string_view = basic_string_view<char32_t>;
using wstring_view = basic_string_view<wchar_t>;

// 21.4.5, hash support
template<class T> struct hash;
template<> struct hash<string_view>;
template<> struct hash<u8string_view>;
template<> struct hash<u16string_view>;
template<> struct hash<u32string_view>;
template<> struct hash<wstring_view>;
[...]

```

Change in 21.4.5 [string.view.hash]:

```

template<> struct hash<string_view>;
template<> struct hash<u8string_view>;
template<> struct hash<u16string_view>;
template<> struct hash<u32string_view>;
template<> struct hash<wstring_view>;

```

Change in table 65 of 22.3.1.1.1 [locale.category]:

Table 65 — Locale category facets

Category	Includes facets
[...]	[...]
ctype	ctype<char>, ctype<wchar_t> codecvt<char, char, mbstate_t> codecvt<char16_t, char, mbstate_t> (deprecated) codecvt<char32_t, char, mbstate_t> (deprecated) codecvt<char16_t, char8_t, mbstate_t> codecvt<char32_t, char8_t, mbstate_t> codecvt<wchar_t, char, mbstate_t>
[...]	[...]

Change in table 66 of 22.3.1.1.2 [locale.facet]:

Table 66 — Required specializatoinis	
Category	Includes facets
[...]	[...]
ctype	ctype_byname<char>, ctype_byname<wchar_t> codecvt_byname<char, char, mbstate_t> codecvt_byname<char16_t, char, mbstate_t> (deprecated) codecvt_byname<char32_t, char, mbstate_t> (deprecated) codecvt_byname<char16_t, char8_t, mbstate_t> codecvt_byname<char32_t, char8_t, mbstate_t> codecvt_byname<wchar_t, char, mbstate_t>
[...]	[...]

Change in 22.4.1.4 [locale.codecvt] paragraph 3:

The specializations required in Table 65 (22.3.1.1.1) convert the implementation-defined native character set. `codecvt<char, char, mbstate_t>` implements a degenerate conversion; it does not convert at all. The specializations `codecvt<char16_t, char, mbstate_t>` (deprecated) and `codecvt<char16_t, char8_t, mbstate_t>` converts between the UTF-16 and UTF-8 encoding forms, and the specializations `codecvt<char32_t, char, mbstate_t>` (deprecated) and `codecvt<char32_t, char8_t, mbstate_t>` converts between the UTF-32 and UTF-8 encoding forms. `codecvt<wchar_t, char, mbstate_t>` converts between the native character sets for narrow ordinary and wide character. Specializations on `mbstate_t` perform conversion between encodings known to the library implementer. Other encodings can be converted by specializing on a user-defined `stateT` type. Objects of type `stateT` can contain any state that is useful to communicate to or from the specialized `do_in` or `do_out` members.

Change in 22.5 [locale.stdcvt] paragraph 2:

Header <codecvt> synopsis

```
namespace std {
    enum codecvt_mode {
        consume_header = 4,
        generate_header = 2,
        little_endian = 1
    };
    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf8
    : public codecvt<Elem, charchar8_t, mbstate_t> {
    public:
        explicit codecvt_utf8(size_t refs = 0);
        ~codecvt_utf8();
    };

    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf16
    : public codecvt<Elem, charchar8_t, mbstate_t> {
    public:
        explicit codecvt_utf16(size_t refs = 0);
        ~codecvt_utf16();
    };

    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf8_utf16
    : public codecvt<Elem, charchar8_t, mbstate_t> {
    public:
        explicit codecvt_utf8_utf16(size_t refs = 0);
        ~codecvt_utf8_utf16();
    };
}
```

```
};
}
```

Change in 27.3 [iostream.forward]:

```
[...]
template<class charT> class char_traits;
template<> class char_traits<char>;
template<> class char_traits<char8_t>;
template<> class char_traits<char16_t>;
template<> class char_traits<char32_t>;
template<> class char_traits<wchar_t>;
[...]
```

Change in 27.10.4.10 [fs.def.native.encode]:

For **narrow ordinary** character strings, the operating system dependent current encoding for pathnames (27.10.4.18).
For wide character strings, the implementation defined execution wide-character set encoding (2.3).

Change in 27.10.5 [fs.req] paragraph 1:

Throughout this sub-clause, char, wchar_t, **char8_t**, char16_t, and char32_t are collectively called *encoded character types*.

Change in 27.10.6 [fs.filesystem.syn]:

```
// 27.10.8.6.2 D.14, path factory functions (deprecated):
template <class Source>
    path u8path(const Source& source);
template <class InputIterator>
    path u8path(InputIterator first, InputIterator last);
```

Change in 27.10.8 [class.path] paragraph 1:

```
[...]
std::string string() const;
std::wstring wstring() const;
std::stringu8string u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;
[...]
```

```
std::string generic_string() const;
std::wstring generic_wstring() const;
std::stringu8string generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;
[...]
```

Add a new subparagraph after 27.10.8.2.2 [fs.req] paragraph 1 (1.2):

— char8_t: The encoding is UTF-8. The method of conversion method is unspecified.

Change in 27.10.8.4.6 [path.native.obs] paragraph 8:

```
std::string string() const;
std::wstring wstring() const;
std::stringu8string u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;
```

Returns: pathname.

Change in 27.10.8.4.6 [path.native.obs] paragraph 9:

Remarks: Conversion, if any, is performed as specified by 27.10.8.2. The encoding of the strings returned by `u8string()`, `u16string()`, and `u32string` are always UTF-8, UTF-16, and UTF-32 respectively.

Change in 27.10.8.4.7 [path.generic.obs] paragraph 5:

```
std::string generic_string() const;
std::wstring generic_wstring() const;
std::string generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;
```

Returns: pathname, reformatted according to the generic pathname format (27.10.8.1).

Change in 27.10.8.4.7 [path.generic.obs] paragraph 6:

Remarks: Conversion, if any, is performed as specified by 27.10.8.2. The encoding of the strings returned by `generic_u8string()`, `generic_u16string()`, and `generic_u32string` are always UTF-8, UTF-16, and UTF-32 respectively.

Change in 27.10.8.6.2 [path.factory] paragraph 1:

Requires: The source and [first, last) sequences are UTF-8 encoded. The value type of Source and InputIterator is `char` or `char8_t`.

Drafting note: It is intentional that the deprecated factory functions accept ranges with value types of either `char` or `char8_t`. This is a backward compatibility feature.

Add a new subparagraph after 27.10.8.6.2 [path.factory] paragraph 2 (2.1):

— If value type is `char8_t`, return `path(source)` or `path(first, last)`; otherwise,

Change in 27.10.8.6.2 [path.factory] paragraph 4:

[*Example:* A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

```
namespace fs = std::filesystem;
std::string utf8_string = read_utf8_data();
fs::create_directory(fs::u8path(utf8_string));
```

Move subclause 27.10.8.6.2 [path.factory] after D.13 [depr.iterator.primitives], renumber to D.14, and rename to [depr.path.factory]

Drafting note: The `u8path` factory functions are deprecated.

Change in 29.2 [atomics.syn]:

```
[...]
// 29.4, lock-free property
#define ATOMIC_BOOL_LOCK_FREE unspecified
#define ATOMIC_CHAR_LOCK_FREE unspecified
#define ATOMIC_CHAR8_T_LOCK_FREE unspecified
#define ATOMIC_CHAR16_T_LOCK_FREE unspecified
#define ATOMIC_CHAR32_T_LOCK_FREE unspecified
#define ATOMIC_WCHAR_T_LOCK_FREE unspecified
[...]
```

Change in 29.4 [atomics.lockfree]:

```
#define ATOMIC_BOOL_LOCK_FREE unspecified
#define ATOMIC_CHAR_LOCK_FREE unspecified
#define ATOMIC_CHAR8_T_LOCK_FREE unspecified
#define ATOMIC_CHAR16_T_LOCK_FREE unspecified
#define ATOMIC_CHAR32_T_LOCK_FREE unspecified
#define ATOMIC_WCHAR_T_LOCK_FREE unspecified
[...]
```

Change in 29.5 [atomics.types.generic] paragraph 4:

There shall be explicit specializations of the atomic template for the integral types char, signed char, unsigned char, short, unsigned short, int, unsigned int, long, unsigned long, long long, unsigned long long, **char8_t**, char16_t, char32_t, wchar_t, and any other types needed by the typedefs in the header <cstdint>. [...]

Change table 134 in 29.5 [atomics.types.generic] paragraph 8:

There shall be atomic typedefs corresponding to non-atomic typedefs as specified in Table 135. atomic_intN_ -

Table 134 — Named atomic types

Named atomic type	Corresponding non-atomic type
[...]	[...]
atomic_char8_t	char8_t
atomic_char16_t	char16_t
atomic_char32_t	char32_t
atomic_wchar_t	wchar_t

Change in A.6 [gram.dcl]:

```
[...]
simple-type-specifier:    [...]
    char
    char8_t
    char16_t
    char32_t
    wchar_t
    [...]
[...]
```

Acknowledgements

Michael Spencer and Davide C. C. Italiano first proposed adding a new char8_t fundamental type in [P0372R0](#) [\[P0372R0\]](#).

References

[N2249] Lawrence Crowl, "New Character Types in C++", N2249, 2007.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2249.html>

[N4197] Richard Smith, "Adding u8 character literals", N4197, 2014.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4197.html>

[N4606] "Working Draft, Standard for Programming Language C++", N4606, 2016.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4606.pdf>

[P0353R0] Beman Dawes, "Unicode Encoding Conversions for the Standard Library", P0353R0, 2016.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0353r0.html>

[P0372R0] Michael Spencer and Davide C. C. Italiano, "A type for utf-8 data", P0372R0, 2016.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0372r0.html>

[P0244R1] Tom Honermann, "Text_view: A C++ concepts and range based character encoding and code point enumeration library", P0244R1, 2016.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0244r1.html>

[P0218R1] Beman Dawes, "Adopt the File System TS for C++17", P0218R1, 2016.
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0372r0.html>